

# 11 Understanding Ambiguity in Requirements Engineering

*Erik Kamsties*

**Abstract:** This chapter illustrates that ambiguity is a serious problem of natural language requirements documents, which is not limited to simple language problems such as multiple referents of an “it”. The results of two empirical studies are presented, which indicate that on one hand ambiguity problems are not solved by formalization during further software development activities, and, on the other hand, it is difficult to detect all ambiguities, even if the reader is aware of all the facets of ambiguity. A combination of the results of both studies indicated that most ambiguities that slip through formalization can be detected by a previous inspection using a tailored reading technique. Based on these results, recommendations are made on how to live with the inevitable ambiguity in the RE process.

**Keywords:** Natural language requirements, Ambiguity, Disambiguation, Inspection, Formalization, Empirical study.

## 11.1 Introduction

In industrial requirements engineering (RE), natural language is the most frequently used representation in which to state requirements that are to be met by information technology products or services. Diagrams and other semi-formal representations are often used to supplement informal requirements specifications. Fully formalized requirements specifications are rare. A recent on-line survey indicates that 79% of the requirements documents are written in common natural language, 16% are written in structured natural language, and only 5% are written in a formalized language [20].

The use of natural language to specify requirements has many benefits, although it bears also some problems, as Chap. 10 has argued. A major, well-recognized problem is the inherent ambiguity of natural language. Stakeholders are often not even aware that there is an ambiguity in a requirement, i.e., the ambiguity is unintentional. Each stakeholder gets from reading the requirements an understanding that differs from that of others, without recognizing this difference. Consequently, the software developers design and implement a system that does not behave as intended by the users, but the developers honestly believe they have followed the requirements. Also, components fail to interact properly, because the same requirement was allocated to different components, and the different developers of the components have interpreted the requirement differently.

Ambiguity is also a feature of natural language. It can be intentionally used, e.g., to postpone decisions, because they are considered design decisions. This chapter focuses on unintentional ambiguity.

We distinguish between *linguistic ambiguity* and *software engineering ambiguity*. The former is context independent and can be observed by any reader who has a tone for language. An example, taken from [25], is:

(1) *The product shall show the weather for the next 24 hours.*

The phrase *for the next twenty-four hours* can be attached to the verb *show* or to the noun *weather*. Thus, the requirement can be interpreted as the product shall show the current weather and continue to do so for the next 24 hours or the product shall show the projected weather for the forthcoming 24 hours.

A software engineering (SE) ambiguity is context dependent and can be observed only by a reader who has knowledge of the particular project's domain. Parnas, Asmis, and Madey give an example of such an ambiguity in a requirement that happens to be about a continually varying water level in a tank [22]:

(2) *Shut off the pumps if the water level remains above 100 meters for more than 4 seconds.*

The authors claim that this type of ambiguity is very common in informal requirements documents. One can find four interpretations: Shut off the pumps if the *mean/median/root mean square/minimum* water level over the past 4 seconds was above 100 meters. However, the software engineers did not notice this ambiguity and quietly assumed the fourth interpretation. Unfortunately, under this interpretation, with sizable rapid waves in the tank, the water level can be dangerously high without triggering the shut off. In general, the interpretation of the ambiguity is very much a function of the reader's background. For example, in many other engineering areas, the standard interpretation would be the third.

SE ambiguities are more important than linguistic ones. Although a requirements sentence may be ambiguous because of multiple word senses, syntactic sentence readings, or referenced items, psycho-linguistic experiments show that there is often one preferred sentence reading after semantics and the context are considered [23]. In the requirements documents that we have investigated, SE ambiguities account for the majority of ambiguities, while purely linguistic ambiguities played a less significant role. The requirements document used throughout the studies described in this chapter contains 4 linguistic but 34 SE ambiguities.

This chapter provides a comprehensive analysis of ambiguity in requirements engineering. First, the related work on how to deal with ambiguity in RE is discussed. Then, a definition of linguistic and SE ambiguity is provided to increase awareness for the multi-facetted issues of ambiguity. Next, the results of an empirical study on the effect of unintended ambiguity on further formalization during the RE process are reported. An ambiguity also conveys the intended meaning, i.e., it is only a *potential* defect and the question is: which types of ambiguity really cause problems? Afterwards, a reading technique tailored to ambiguity and empirical results of its application are presented. Finally, we show that formalization accompanied with inspection in the RE process can lead to the conscious